Computational Graph

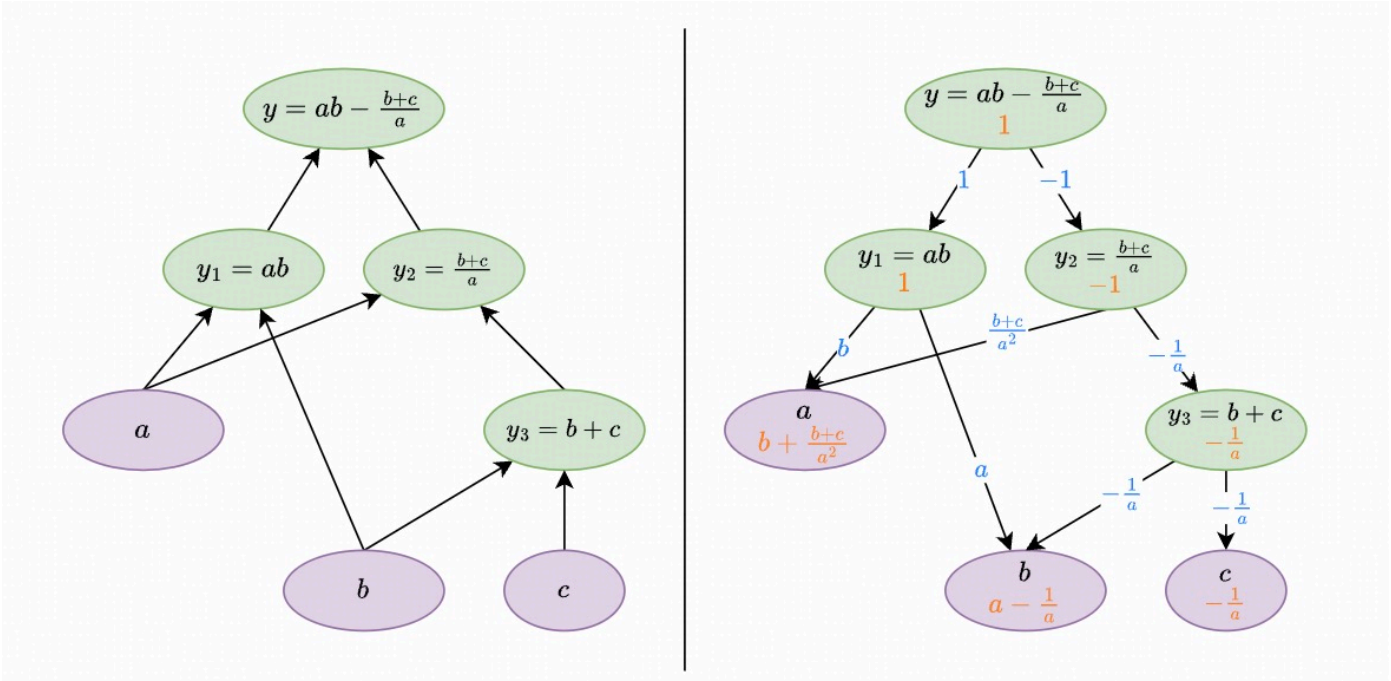
Problem Description

A computational graph is a directed acyclic graph used to represent mathematical computations. In a computational graph, nodes represent operations (such as addition, subtraction, multiplication, division, etc.) or variables (such as inputs), and edges represent dependencies between these operations or variables. An important application of computational graphs is automatic differentiation. Using the chain rule (if $y = f(y_1(x), y_2(x), \cdots, y_n(x))$, then $\frac{\mathrm{d}y}{\mathrm{d}x} = \sum_{i=1}^{n} \frac{\mathrm{d}y_i}{\mathrm{d}x} \cdot \frac{\partial y}{\partial y_i}$), computational graphs can support backpropagation algorithms, efficiently computing gradients (i.e., derivatives), which are used to train neural networks.

A computational graph contains two types of nodes: leaf nodes and non-leaf nodes. Leaf nodes have a numerical value and need to compute their gradient with respect to a specified node during backpropagation; non-leaf nodes are obtained by performing various computations between nodes, have several input nodes and one output value, and are thus also called operators. The entire algorithm is divided into two parts:

- The first part is the forward computation process, where various operations are performed between nodes to ultimately obtain a node as the result.
- The second part is the backpropagation process, where the gradient is propagated starting from the result node obtained in the first part:
    - For non-leaf nodes, if their gradient with respect to the result node is g, they need to compute the gradient of each input node with respect to the result node and recursively propagate the gradient to these input nodes;
    - For leaf nodes, they only need to accumulate the gradients propagated from all non-leaf nodes;
    - At the start of the algorithm, the gradient propagation begins from the result node, and evidently, the gradient of the result node is 1 (because $\frac{\mathrm{d}y}{\mathrm{d}y} = 1$).

An example of a computational graph is shown below. The left diagram represents the forward computation process, and the right diagram represents the backpropagation process. The orange numbers indicate the gradient of the node with respect to y, and the blue numbers indicate the gradient propagation. Eventually, we can obtain $\frac{\partial y}{\partial a} = b + \frac{b+c}{a^2}$, $\frac{\partial y}{\partial b} = a - \frac{1}{a}$, $\frac{\partial y}{\partial c} = -\frac{1}{a}$.



You only need to implement four operators: addition, subtraction, multiplication, and division. Their backpropagation computation have already been provided:

- For $y = a + b$, if the gradient of y is g, then the gradients of a and b are both g.
- For $y = a - b$, if the gradient of y is g, then the gradient of a is g, and the gradient of b is $-g$.
- For $y = ab$, if the gradient of y is g, then the gradient of a is gb, and the gradient of b is ga.
- For $y = \frac{a}{b}$, if the gradient of y is g, then the gradient of a is $\frac{g}{b}$, and the gradient of b is $-\frac{ga}{b^2}$.

There is some code in `leaf_node.h`, `leaf_node.cpp`, `node.h`, and `node.cpp`. Please read and complete these files to implement the functionality of a simple computational graph. Of course, you can also modify these files as needed to add interfaces or new classes. **Your code must ensure there are no memory leaks.**

Here are the interfaces you need to implement:

- `Node` class, representing non-leaf nodes:
  - `float get_val()`, returns a float representing the value of the node. This interface is called during the first part of the algorithm to get the forward computation result;
  - `void backward(float g)`, given that the gradient of this node is g, perform backpropagation based on the type of operator. This interface is called on the result node during the second part of the algorithm using `backward(1.0)`.
- `LeafNode` class, representing leaf nodes.
- `+-*/` operators.

Download problem files

Test Data and Constraints

This problem has 10 test points, each worth 10 points. Each test point has a separate `main.cpp` file, named `main0.cpp` to `main9.cpp`, and all test points have no input.

The constraints for each test point are as follows:

- Test points 0, 1, 2: only involve addition and subtraction, only include forward computation, do not check for memory leaks.
- Test points 3, 4, 5: only involve addition and subtraction, do not check for memory leaks.
- Test points 6, 7: do not check for memory leaks.
- Test points 8, 9: no special constraints.

Test points 0, 3, 6, and 8 include `main.cpp` files and corresponding answers as examples in the provided files.

The scale of the computational graph involved in all test points is small enough that you do not need to specifically focus on the runtime speed of the program.

Note: When calling `backward(float)`, it is guaranteed that the lifecycle of all involved `LeafNode` instances is not over. You can assume that the following type of code will not appear in the test data:

```
LeafNode a(3.0);
Node b = a;
if (true) {
  LeafNode c(5.0);
  b = b + c;
}
b.backward(1.0);
```

Since `c` constructed within the braces would have been destructed by the time `b.backward(1.0)` is called, this code is illegal. Similarly, here is another example of illegal code:

```
Node b = LeafNode(2.33);
b.backward(1.0);
```

Hints

All hints are not mandatory:

1. Consider implementing implicit type conversion from `LeafNode` to `Node`.
2. Design the `Node` class as a wrapper that holds a pointer to the actual operator. This could be beneficial: operations between `Node` instances are not pointer-based, making it inconvenient to directly implement polymorphism. However, implementing polymorphism via pointers stored inside the class is very convenient.
3. Multiplication and division are not very different from addition and subtraction. Once you complete the first six test points, you might quickly pass test points 6 and 7.
4. You might use smart pointers for memory management, but you might need to avoid smart pointers accidentally destructing `LeafNode`.

Submission Format

Please package `leaf_node.h`, `leaf_node.cpp`, `node.h`, and `node.cpp` into a zip file and upload it. Other submitted files will be ignored. **Note: Your files should be in the root directory of the zip file, not in a subfolder. In other words, when your zip file is extracted, it should directly yield a series of cpp files, h files, and other code files, not a folder containing them.** During evaluation, the OJ will insert the provided files into your directory for compilation and execution.